Code architecture: the repository

How the folders should look · the coloured number is the member who owns the file

1 Data, Sentinel-2 and integration · 2 Signal processing, response and product · 3 Trust, attacks and threat model · 4 Detection and evaluation · all shared

.		
├── README.md	Quick start and the one-command reproduction	1
├── requirements.txt	Pinned versions of every dependency	1
├── pyproject.toml	Makes the package installable with pip install -e .	1
├── .gitignore	Keeps results, raw data, keys and credentials out of git	1
├── **docs/**	The planning documents (already written)	all
├── **config/**		
│ └── variables.yaml	Variable names, units, POWER parameters, physical limits	2
├── **scenarios/**	Every experiment is a YAML file here, never a code edit	
│ ├── sites.yaml	Development sites and held-out sites, with coordinates	1
│ ├── **standard/**	step_12pct.yaml, drift_15pct.yaml, replay_7d.yaml	3
│ ├── **injection/**	s00_clean.yaml to s11_key_compromise.yaml	3
│ └── **evasion/**	v1_slow_drift.yaml to v9_whitebox.yaml	3
├── **src/sensorguard/**	The package: see the next page	all
├── **scripts/**		
│ ├── fetch_data.py	Download, cache and verify the datasets	1
│ ├── assess.py	File mode: series and coordinates in, trust scores out	1
│ └── run_all.py	Reproduce every table and figure (quick and full mode)	1
├── **tests/**	One test file per module, run with pytest	
│ ├── test_data.py		1
│ ├── test_processing.py		2
│ ├── test_trust.py	Forged, modified, replayed and missing records	3
│ ├── test_attacks.py	Same seed gives the same series and labels	3
│ ├── test_detect.py		4
│ └── test_pipeline.py	End to end on one site in quick mode	1
├── **data/**		
│ ├── **cache/**	Cached reference data with hashes (small, committed)	1
│ └── **raw/**	Downloaded station files (not committed)	1
├── **keys/**	Generated station and root keys (not committed)	3
├── **results/**	Generated tables and plots (not committed)	4
└── **report/**		
├── **sections/**	One file per report section, written by its owner	all
└── **figures/**	Final figures copied from results/	4

How data flows through the package

Stream mode (signed records, one per day):

```
data → sensor → transport → trust → processing → detect → response → product
```

File mode (the jury's series and coordinates):

```
data → processing → detect → response → product
```

`attacks` can be inserted at any arrow. `eval` runs the whole chain from a scenario file.

Rules that keep four people from colliding

1. Each stage is plain functions over tables and arrays, so it can be tested alone.
2. A module imports only `schema` and the stages before it. Only `pipeline` and `eval` import everything.
3. Change `schema.py` only with the agreement of all four.
4. An experiment is a YAML file in `scenarios/`, never an edit to the code.
5. Never committed: `results/`, `data/raw/`, `keys/`, any credential.
6. No names in any file, path or comment.

Added to the layout in docs/02

`schema.py`, `pipeline.py`, `pyproject.toml`, `config/variables.yaml`, `keys/`, `data/raw/`, and the file names inside each folder. The folder names are unchanged.

Code architecture: the package

`src/sensorguard/` · one folder per stage, one owner per file · `sensorguard` is a neutral placeholder name

1 Data, Sentinel-2 and integration · 2 Signal processing, response and product · 3 Trust, attacks and threat model · 4 Detection and evaluation · all shared

src/sensorguard/		
schema.py	The four agreed formats, in one place	all
pipeline.py	Wires the stages for file mode and stream mode	1
data/		
power.py	NASA POWER client with on-disk cache	1
stations.py	Loaders for the public station archives	1
sites.py	Site list, extreme-weather and fault labels	1
sentinel2.py	Small windows: cloud, snow, NDVI, NDMI, NDWI	1
cache.py	Checksums and cache lookup	1
sensor/		
station.py	Emulated station: replays and signs real readings	3
trust/		
keys.py	Root key, station keys, SecureElement interface	3
enrolment.py	Certificates, registry, revocation	3
record.py	Signed record format, encode and decode	3
verify.py	Verification, replay state, cadence watchdog	3
ledger.py	Hash-chained, tamper-evident log	3
transport/		
link.py	Loss, delay, duplication, reordering	3
attacks/		
official.py	12 % step, drift to 15 %, 7-day replay	3
injections.py	The rest of the suite: freeze, masking, noise	3
evasion.py	Evasions E1 to E9	3
labels.py	Exact per-day labels for every injection	3

src/sensorguard/ (continued)		
processing/		
qc.py	The seven quality-control tests	2
align.py	Day-boundary lag and gap marking	2
calibrate.py	Robust per-season map; cold-start mode	2
residuals/		
temporal.py	R1: season and the station's own past	2
physical.py	R2: physics between variables	2
reference.py	R3: against NASA POWER, two forms	2
orbital.py	R4: against Sentinel-2 indicators	1
replay.py	R5: replay features	2
detect/		
statistical.py	Detector S: running sum, change points, trend	4
learned.py	Detector M: the chosen PyOD one-class model	4
supervised.py	Random forest, upper-bound reference only	4
fusion.py	Scores to a calibrated compromise probability	4
trust_score.py	Station trust score and trust bands	4
response/		
policy.py	De-weight, verify, quarantine, alert	2
product/		
water_balance.py	FAO-56, 30-day balance, decision levels	2
provenance.py	Signed product with its provenance record	2
eval/		
runner.py	Runs a scenario file end to end	4
metrics.py	Detection rate, latency, onset error, false alarms	4
resources.py	Time and memory measurement	4
figures.py	Every plot of the report	4